



Department of Artificial Intelligence and Machine Learning

Experiment No.: 6 To implement Skip List

Aim

To implement a Skip List and perform insertion, searching, deletion, and traversal operations using probabilistically generated levels.

Learning Objectives

- To understand the structure and working of a Skip List.
- To understand how multiple levels improve searching efficiency.
- To study the probabilistic method used for assigning levels to newly inserted elements.
- To implement insertion, searching, and deletion operations.
- To analyze the expected time complexity of Skip List operations.

Theory

A **Skip List** is a probabilistic data structure used to store elements in sorted order. It provides efficient searching, insertion, and deletion operations.

A Skip List consists of multiple levels of linked lists. The lowest level contains all the elements, while the higher levels contain only selected elements. These higher levels act as express lanes and allow the search operation to skip several elements at a time.

The structure can be visualized as follows:

Level 3: HEAD -----> 40 -----> NIL

Level 2: HEAD -----> 20 -----> 40 -----> NIL

Level 1: HEAD --> 10 -> 20 -----> 40 -> 50 --> NIL

Level 0: HEAD --> 10 -> 20 -> 30 -> 40 -> 50 --> NIL



Searching starts from the highest level. The algorithm moves forward while the next element is smaller than the required key. When further movement is not possible, it moves down one level.

This process continues until the required element is found or the lowest level is reached.

Level Assignment in Skip List

The level of a newly inserted node is generally determined **randomly**.

A probability p is selected, commonly

$$p = 0.5$$

Every node is always inserted at **Level 0**. The node is promoted to higher levels based on repeated random trials.

For example,

$$P(\text{Level} \geq 1) = \frac{1}{2}$$

$$P(\text{Level} \geq 2) = \frac{1}{4}$$

$$P(\text{Level} \geq 3) = \frac{1}{8}$$

Thus, the number of nodes decreases approximately by half as we move towards higher levels.

The maximum level can be selected based on the expected number of elements. A commonly used estimate is

$$L_{\max} \approx \log_{\frac{1}{p}} n$$

where n is the expected number of elements.

For $p = 0.5$,

$$L_{\max} \approx \log_2 n$$

Random Level Generation

A simple random level generation procedure is:

level = 0



```
while random() < p and level < MAX_LEVEL:  
    level = level + 1  
  
return level
```

For example, suppose the generated random numbers are:

0.31, 0.42, 0.76

For $p = 0.5$:

- $0.31 < 0.5 \Rightarrow$ promote to Level 1.
- $0.42 < 0.5 \Rightarrow$ promote to Level 2.
- $0.76 > 0.5 \Rightarrow$ stop.

Therefore, the new node is assigned **Level 2** and participates in Levels 0, 1, and 2.

Algorithm for Searching

1. Start from the header node at the highest level.
2. Compare the next node's key with the search key.
3. If the next key is smaller than the search key, move forward at the same level.
4. If the next key is greater than or equal to the search key, move down one level.
5. Repeat until Level 0 is reached.
6. If the next node contains the required key, report that the element is found.
7. Otherwise, report that the element is not present.

Algorithm for Insertion

1. Start searching from the highest level.
2. Maintain an `update[]` array to store the predecessor of the new node at each level.
3. Move forward while the next node contains a key smaller than the key to be inserted.
4. Move downward until Level 0 is reached.
5. Check whether the key is already present.



6. Generate a random level for the new node.
7. Create the new node with the generated level.
8. Update the forward pointers using the nodes stored in the `update[]` array.

Algorithm for Deletion

1. Start from the highest level of the Skip List.
2. Locate the predecessors of the node to be deleted and store them in the `update[]` array.
3. Move to Level 0 and check whether the required node exists.
4. If the node exists, modify all forward pointers that point to the node.
5. Delete the node.
6. If the highest level becomes empty, reduce the current maximum level.

Illustrative Example

Consider inserting the following elements:

10, 20, 30, 40, 50

Suppose the randomly generated levels are:

Element	Generated Level
10	1
20	2
30	0
40	3
50	1

The resulting Skip List is:

Level 3: HEAD -----> 40 -----> NIL

Level 2: HEAD -----> 20 -----> 40 -----> NIL

Level 1: HEAD --> 10 -> 20 -----> 40 -> 50 --> NIL

Level 0: HEAD --> 10 -> 20 -> 30 -> 40 -> 50 --> NIL



Example: Searching for 30

Suppose the Skip List is searched for key 30.

The search starts from the highest level.

Level 3:

HEAD -> 40

40 > 30

Move DOWN

Level 2:

HEAD -> 20

20 < 30

Move RIGHT to 40

20 -> 40

40 > 30

Move DOWN

Level 1:

20 -> 40

40 > 30

Move DOWN

Level 0:

20 -> 30

30 = Search Key

Element Found

Thus, instead of sequentially examining every node, the higher levels help the search skip unnecessary nodes.

Program

Write a program to implement a Skip List supporting the following operations:

1. Insert an element
2. Search for an element



3. Delete an element
4. Display the Skip List level by level

Expected Output

```
----- Skip List Operations -----
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter choice: 1
Enter element: 10
Element inserted successfully.
Enter choice: 1
Enter element: 20
Element inserted successfully.
Enter choice: 1
Enter element: 30
Element inserted successfully.
Enter choice: 1
Enter element: 40
Element inserted successfully.
Enter choice: 1
Enter element: 50
Element inserted successfully.
Enter choice: 4
Skip List:
Level 3: 40
Level 2: 20 40
Level 1: 10 20 40 50
Level 0: 10 20 30 40 50
Enter choice: 2
Enter element to search: 30
Element 30 found.
Enter choice: 3
Enter element to delete: 30
Element 30 deleted successfully.
```



Level 0: 10 20 40 50

Problem Statement

3417. Zigzag Grid Traversal With Skip

Easy Topics Companies

You are given an $m \times n$ 2D array `grid` of **positive** integers.

Your task is to traverse `grid` in a **zigzag** pattern while skipping every **alternate** cell.

Zigzag pattern traversal is defined as following the below actions:

- Start at the top-left cell $(0, 0)$.
- Move *right* within a row until the end of the row is reached.
- Drop down to the next row, then traverse *left* until the beginning of the row is reached.
- Continue **alternating** between right and left traversal until every row has been traversed.

Note that you **must skip** every *alternate* cell during the traversal.

Return an array of integers `result` containing, **in order**, the value of the cells visited during the zigzag traversal with skips.

Example 1:

Input: `grid = [[1,2],[3,4]]`

Output: `[1,4]`

Explanation:

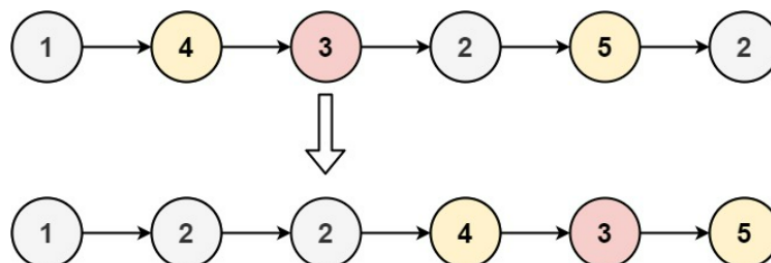
86. Partition List

Medium Topics Companies

Given the `head` of a linked list and a value `x`, partition it such that all nodes **less than** `x` come before nodes **greater equal to** `x`.

You should **preserve** the original relative order of the nodes in each of the two partitions.

Example 1:



Input: `head = [1,4,3,2,5,2], x = 3`

Output: `[1,2,2,4,3,5]`

Example 2:

Input: `head = [2,1], x = 2`

Output: `[1,2]`



Post-Lab Questions

1. What is a Skip List? How does it differ from a normal linked list?
2. What is the purpose of maintaining multiple levels in a Skip List?
3. How is the level of a newly inserted node determined?
4. Why is the level assignment in a Skip List considered probabilistic?
5. If $p = 0.5$, what is the probability that a node reaches Level 3?
6. Explain the role of the `update[]` array during insertion.
7. Why does searching begin from the highest level rather than Level 0?
8. What is the expected time complexity of searching in a Skip List?
9. What is the worst-case time complexity of searching in a Skip List? Under what condition can this occur?
10. Construct a Skip List for the keys

15, 25, 35, 45, 55, 65

assuming their generated levels are

1, 3, 0, 2, 1, 0

respectively. Show the Skip List at every level.

References

1. William Pugh, "Skip Lists: A Probabilistic Alternative to Balanced Trees," *Communications of the ACM*, Vol. 33, No. 6, 1990.
2. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, MIT Press.
3. Mark Allen Weiss, *Data Structures and Algorithm Analysis*, Pearson Education.